

```
In [1]: import sys
print(sys.executable)
```

```
/opt/miniconda3/envs/215a/bin/python
```

```
In [2]: import pandas as pd
import matplotlib.pyplot as plt
```

```
In [3]: USArrests = pd.read_csv("../data/USArrests.csv")
stateCoord = pd.read_csv("../data/stateCoord.csv")
```

```
In [4]: USArrests.head()
```

```
Out[4]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>State</th> <th>Murder</th> <th>Assault</th> <th>UrbanPop</th>
<th>Rape</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th>
<th>4</th> </tbody> <tbody></tbody>
```

Alabama	13.2	236	58	21.2
---------	------	-----	----	------

Alaska	10.0	263	48	44.5
--------	------	-----	----	------

Arizona	8.1	294	80	31.0
---------	-----	-----	----	------

Arkansas	8.8	190	50	19.5
----------	-----	-----	----	------

California	9.0	276	91	40.6
------------	-----	-----	----	------

```
In [5]: stateCoord.head()
```

```
Out[5]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>State</th> <th>lon</th> <th>lat</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>
```

Alabama	-86.592392	34.735676
---------	------------	-----------

Alaska	-168.187471	53.330810
--------	-------------	-----------

Arizona	-111.615136	36.246377
---------	-------------	-----------

Arkansas	-92.240455	36.374502
----------	------------	-----------

California	-120.386476	34.034180
------------	-------------	-----------

```
In [6]: USArrests.columns
```

```
Out[6]: Index(['State', 'Murder', 'Assault', 'UrbanPop', 'Rape'], dtype='str')
```

```
In [7]: stateCoord.columns
```

```
Out[7]: Index(['State', 'lon', 'lat'], dtype='str')
```

```
In [8]: arrests = pd.merge(USArrests, stateCoord, on="State")
```

```

-----
KeyError                                Traceback (most recent call last)
Cell In[8], line 1
----> 1 arrests = pd.merge(USArrests, stateCoord, on="State")

File /opt/miniconda3/envs/215a/lib/python3.14/site-packages/pandas/core/reshape/merge.py:385, in merge(left, right, how, on, left_on, right_on, left_index, right_index, sort, suffixes, copy, indicator, validate)
    371         return _cross_merge(
    372             left_df,
    373             right_df,
    (...) 382             validate=validate,
    383         )
    384     else:
--> 385         op = _MergeOperation(
    386             left_df,
    387             right_df,
    388             how=how,
    389             on=on,
    390             left_on=left_on,
    391             right_on=right_on,
    392             left_index=left_index,
    393             right_index=right_index,
    394             sort=sort,
    395             suffixes=suffixes,
    396             indicator=indicator,
    397             validate=validate,
    398         )
    399         return op.get_result()

File /opt/miniconda3/envs/215a/lib/python3.14/site-packages/pandas/core/reshape/merge.py:1018, in _MergeOperation.__init__(self, left, right, how, on, left_on, right_on, left_index, right_index, sort, suffixes, indicator, validate)
    1008         raise MergeError(msg)
    1010 self.left_on, self.right_on = self._validate_left_right_on(left_on,
    right_on)
    1012 (
    1013     self.left_join_keys,
    1014     self.right_join_keys,
    1015     self.join_names,
    1016     left_drop,
    1017     right_drop,
-> 1018 ) = self._get_merge_keys()
    1020 if left_drop:
    1021     self.left = self.left._drop_labels_or_levels(left_drop)

File /opt/miniconda3/envs/215a/lib/python3.14/site-packages/pandas/core/reshape/merge.py:1620, in _MergeOperation._get_merge_keys(self)
    1618 rk = cast(Hashable, rk)
    1619 if rk is not None:
-> 1620     right_keys.append(right._get_label_or_level_values(rk))
    1621 else:
    1622     # work-around for merge_asof(right_index=True)
    1623     right_keys.append(right.index._values)

```

```

File /opt/miniconda3/envs/215a/lib/python3.14/site-packages/pandas/core/generics.py:1764, in NDFrame._get_label_or_level_values(self, key, axis)
    1760         values = self.xs(key, axis=first_other_axes)._values
    1761     elif self._is_level_reference(key, axis=axis):
    1762         values = self.axes[axis].get_level_values(key)._values
    1763     else:
-> 1764         raise KeyError(key)
    1765
    1766     # Check for duplicates
    1767     if values.ndim > 1:

```

KeyError: 'State'

```
In [9]: stateCoord.columns = stateCoord.columns.str.strip()
```

```
In [10]: stateCoord.columns
```

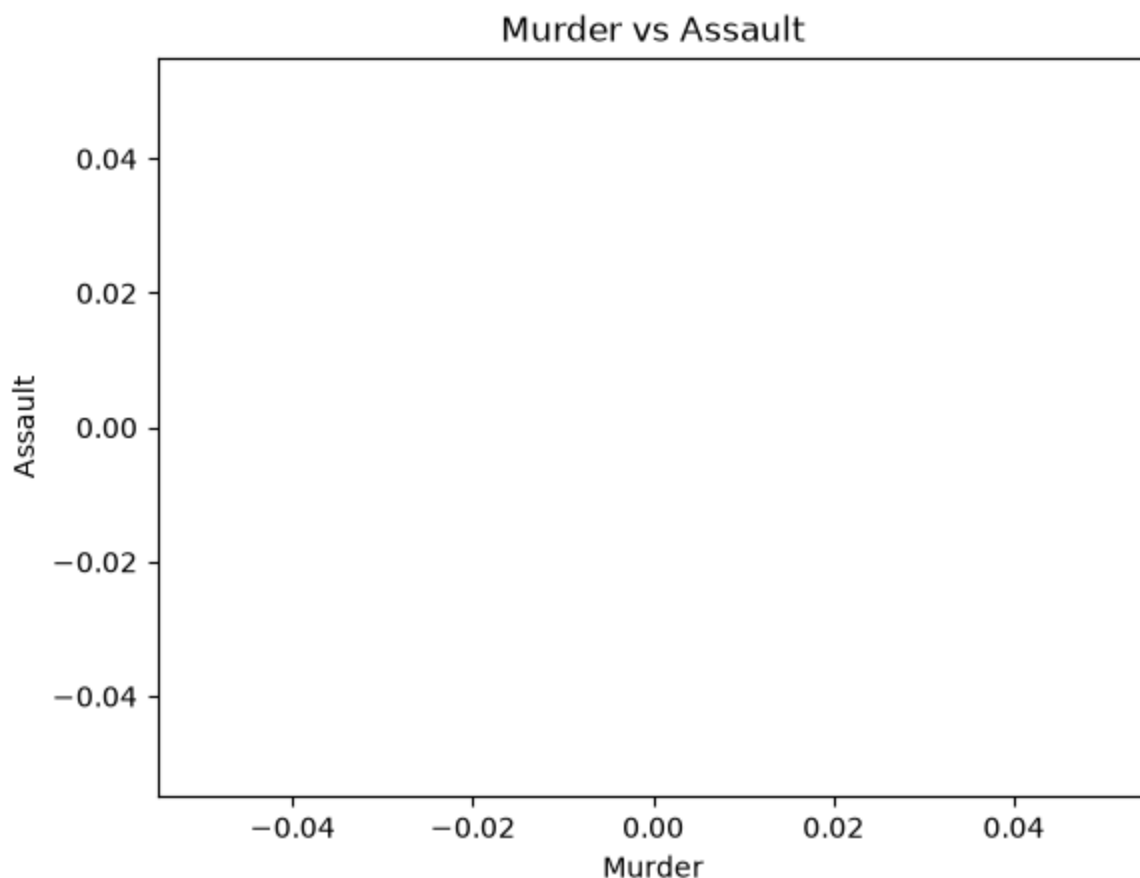
```
Out[10]: Index(['State', 'lon', 'lat'], dtype='str')
```

```
In [11]: arrests = pd.merge(USArrests, stateCoord, on="State")
```

```
In [12]: arrests.head()
```

```
Out[12]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>State</th> <th>Murder</th> <th>Assault</th> <th>UrbanPop</th>
<th>Rape</th> <th>lon</th> <th>lat</th> </thead> <tbody> </tbody> <tbody></tbody>
```

```
In [13]: plt.scatter(arrests["Murder"], arrests["Assault"])
plt.xlabel("Murder")
plt.ylabel("Assault")
plt.title("Murder vs Assault")
plt.show()
```



```
In [14]: USArrests["State"] = USArrests["State"].str.strip()
stateCoord["State"] = stateCoord["State"].str.strip()
```

```
In [15]: arrests = pd.merge(USArrests, stateCoord, on="State")
```

```
In [16]: arrests.head()
```

```
Out[16]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>State</th> <th>Murder</th> <th>Assault</th> <th>UrbanPop</th>
<th>Rape</th> <th>lon</th> <th>lat</th> </thead> <tbody> <th>0</th> <th>1</th>
<th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>
```

Alabama	13.2	236	58	21.2	-86.592392	34.735676
---------	------	-----	----	------	------------	-----------

Alaska	10.0	263	48	44.5	-168.187471	53.330810
--------	------	-----	----	------	-------------	-----------

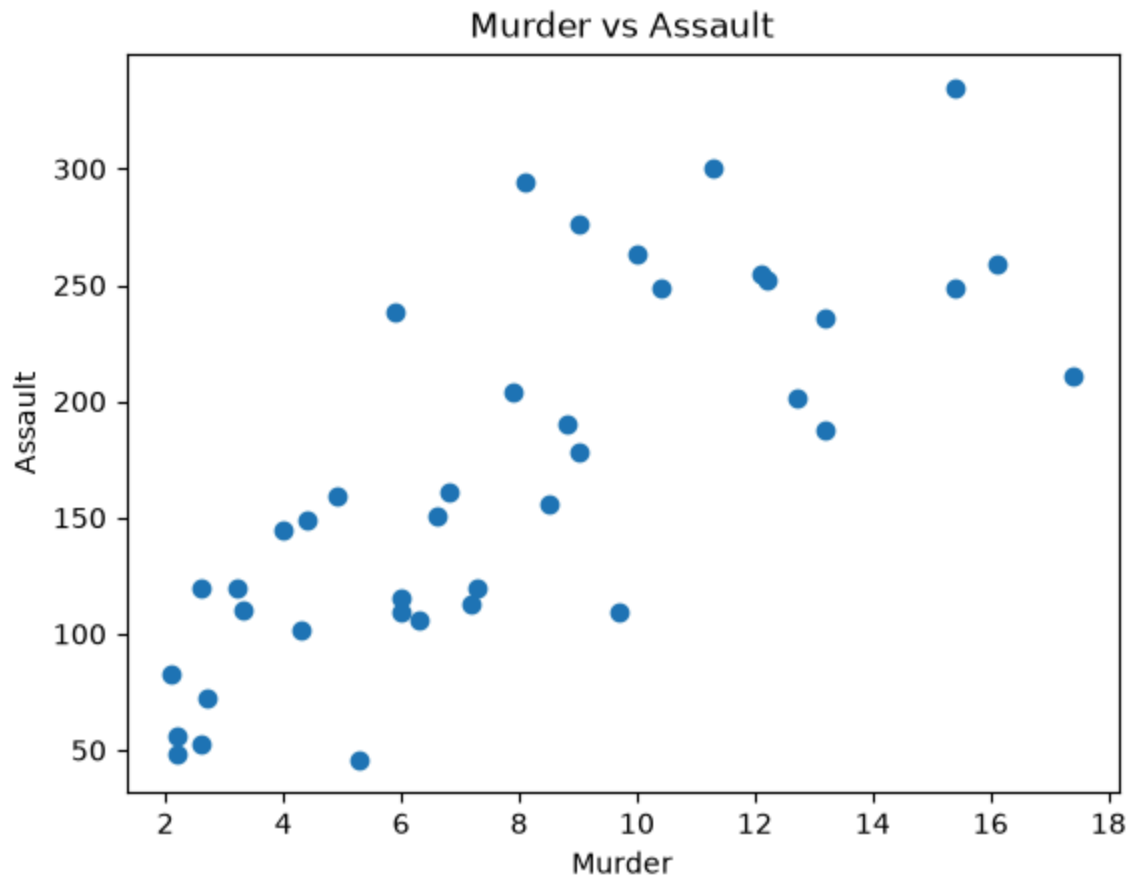
Arizona	8.1	294	80	31.0	-111.615136	36.246377
---------	-----	-----	----	------	-------------	-----------

Arkansas	8.8	190	50	19.5	-92.240455	36.374502
----------	-----	-----	----	------	------------	-----------

California	9.0	276	91	40.6	-120.386476	34.034180
------------	-----	-----	----	------	-------------	-----------

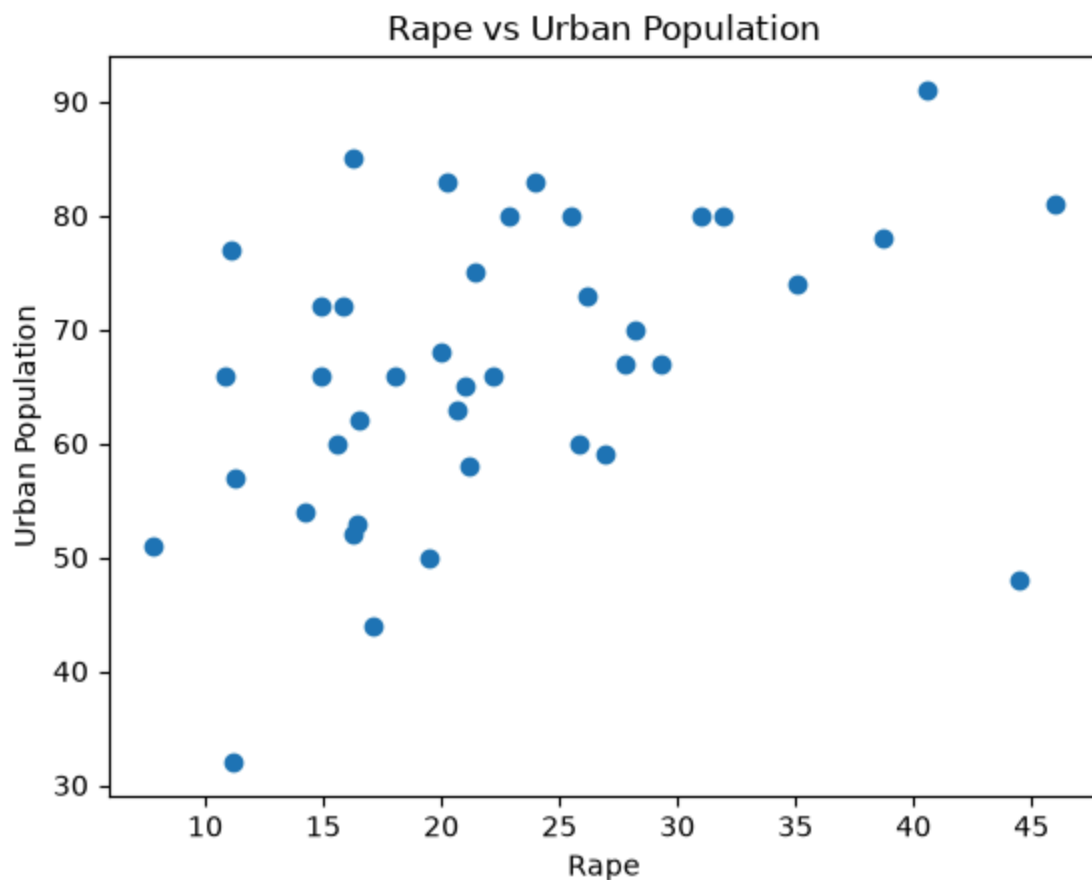
```
In [17]: plt.scatter(arrests["Murder"], arrests["Assault"])
plt.xlabel("Murder")
plt.ylabel("Assault")
```

```
plt.title("Murder vs Assault")  
plt.show()
```



1.3.1. positive relationship between murder and assault rates.

```
In [19]: plt.scatter(arrests["Rape"], arrests["UrbanPop"])  
plt.xlabel("Rape")  
plt.ylabel("Urban Population")  
plt.title("Rape vs Urban Population")  
plt.show()
```



```
In [20]: arrests.sort_values("Rape", ascending=False).head()
```

```
Out[20]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>State</th> <th>Murder</th> <th>Assault</th> <th>UrbanPop</th>
<th>Rape</th> <th>lon</th> <th>lat</th> </thead> <tbody> <th>27</th> <th>1</th>
<th>4</th> <th>5</th> <th>21</th> </tbody> <tbody></tbody>
```

Nevada	12.2	252	81	46.0	-117.034132	39.656972
Alaska	10.0	263	48	44.5	-168.187471	53.330810
California	9.0	276	91	40.6	-120.386476	34.034180
Colorado	7.9	204	78	38.7	-105.837605	40.112391
Michigan	12.1	255	74	35.1	-85.553712	45.662505

```
In [21]: outlier = arrests["State"] == "Alaska"
```

```
plt.scatter(
    arrests.loc[~outlier, "Rape"],
    arrests.loc[~outlier, "UrbanPop"],
    label="Other states"
)

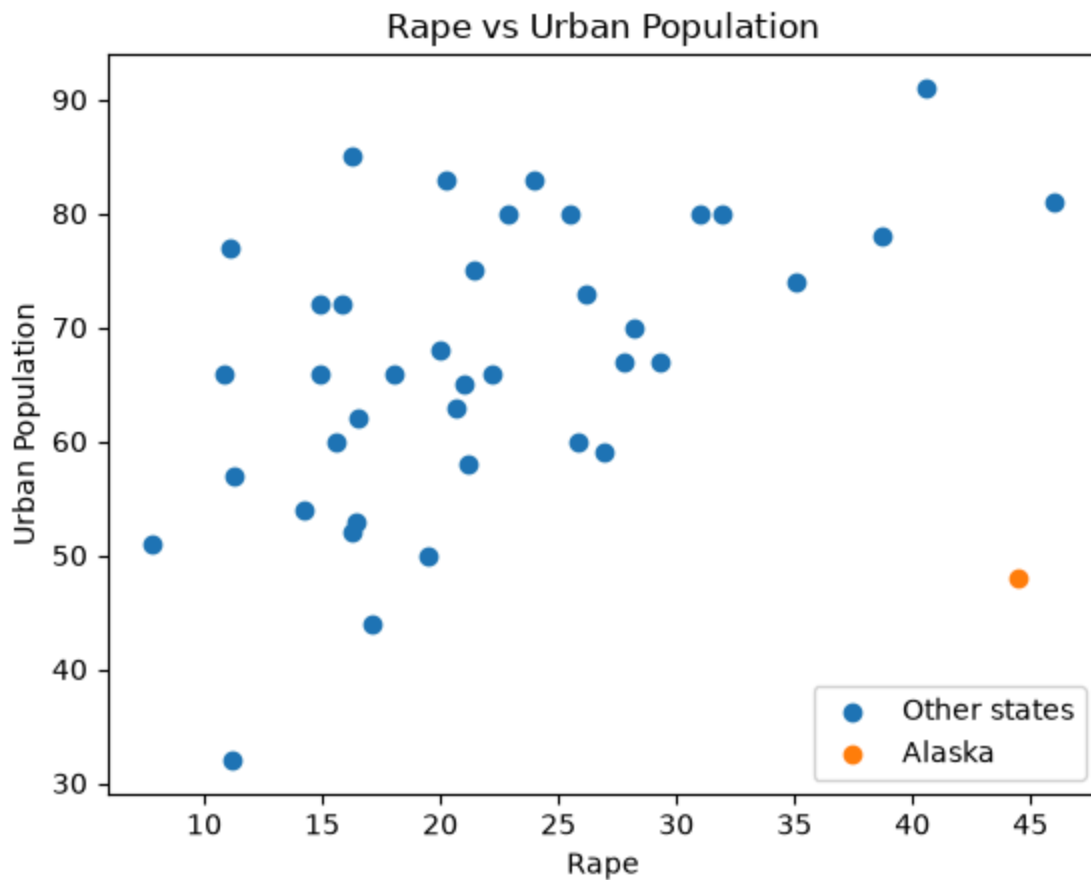
plt.scatter(
    arrests.loc[outlier, "Rape"],
```

```

arrests.loc[outlier, "UrbanPop"],
label="Alaska"
)

plt.xlabel("Rape")
plt.ylabel("Urban Population")
plt.title("Rape vs Urban Population")
plt.legend()
plt.show()

```



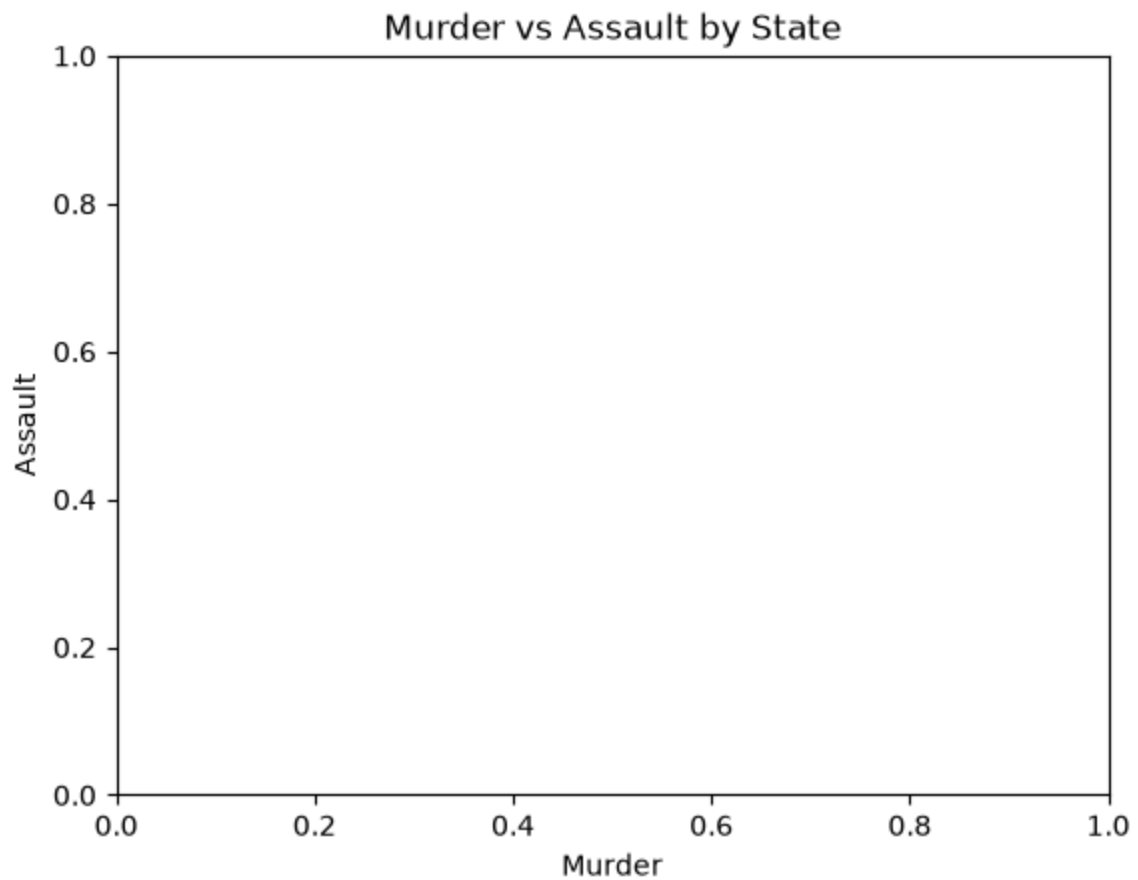
Alaska appears to be an outlier

```

In [22]: for _, row in arrests.iterrows():
plt.annotate(
    row["State"],
    (row["Murder"], row["Assault"]),
    fontsize=8
)

plt.xlabel("Murder")
plt.ylabel("Assault")
plt.title("Murder vs Assault by State")
plt.show()

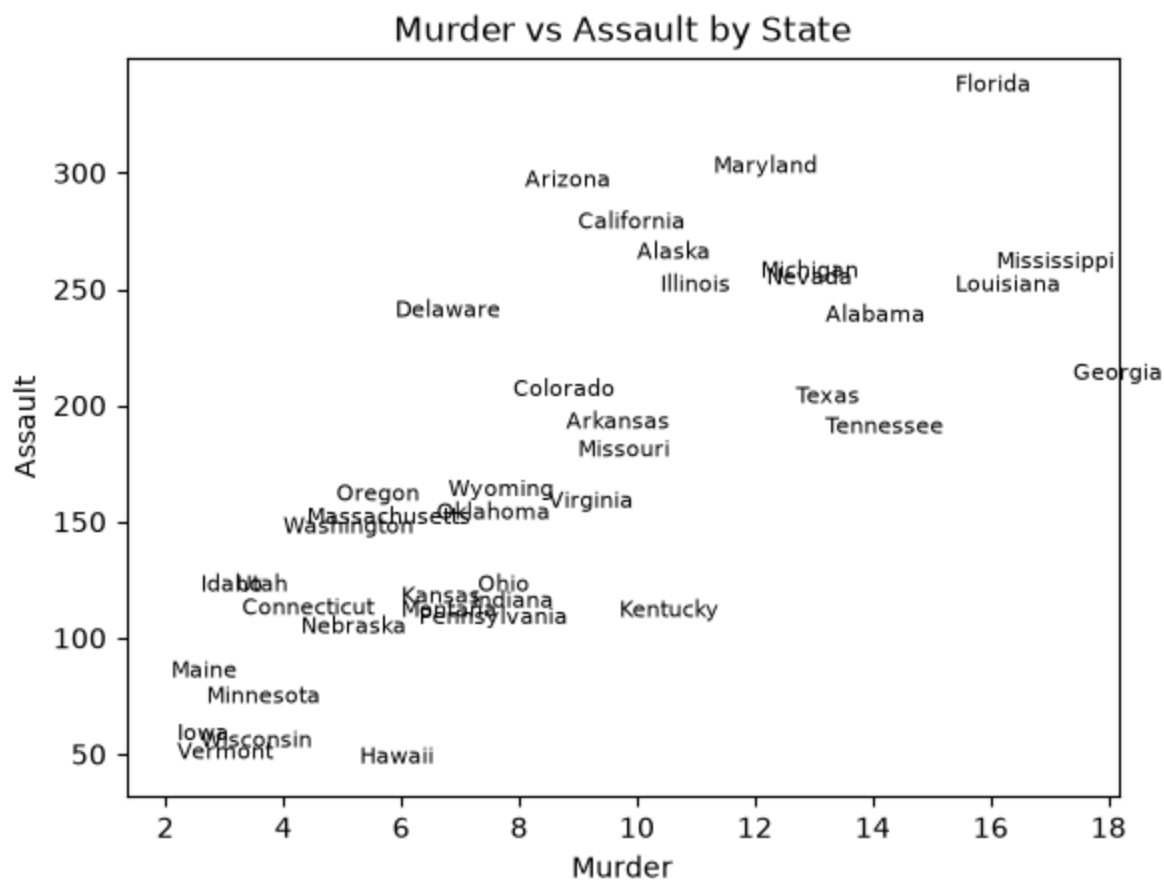
```



```
In [23]: plt.scatter(arrests["Murder"], arrests["Assault"], alpha=0)

for _, row in arrests.iterrows():
    plt.annotate(
        row["State"],
        (row["Murder"], row["Assault"]),
        fontsize=8
    )

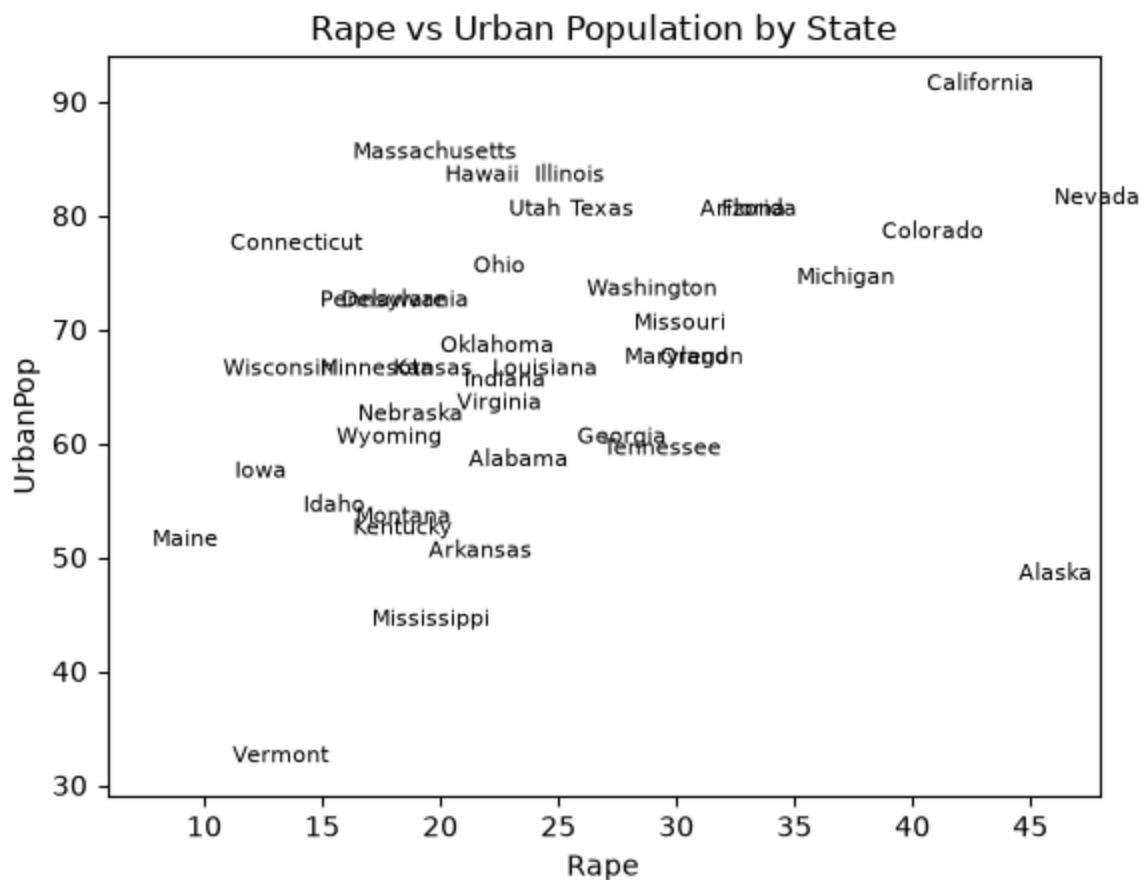
plt.xlabel("Murder")
plt.ylabel("Assault")
plt.title("Murder vs Assault by State")
plt.show()
```



```
In [25]: plt.scatter(arrests["Rape"], arrests["UrbanPop"], alpha=0)

for _, row in arrests.iterrows():
    plt.annotate(
        row["State"],
        (row["Rape"], row["UrbanPop"]),
        fontsize=8
    )

plt.xlabel("Rape")
plt.ylabel("UrbanPop")
plt.title("Rape vs Urban Population by State")
plt.show()
```



```
In [26]: reg_data = arrests.drop(columns=["Murder", "Assault"])
```

```
In [27]: reg_data.head()
```

```
Out[27]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>State</th> <th>UrbanPop</th> <th>Rape</th> <th>lon</th> <th>lat</th>
</thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody>
<tbody></tbody>
```

Alabama	58	21.2	-86.592392	34.735676
---------	----	------	------------	-----------

Alaska	48	44.5	-168.187471	53.330810
--------	----	------	-------------	-----------

Arizona	80	31.0	-111.615136	36.246377
---------	----	------	-------------	-----------

Arkansas	50	19.5	-92.240455	36.374502
----------	----	------	------------	-----------

California	91	40.6	-120.386476	34.034180
------------	----	------	-------------	-----------

```
In [28]: from sklearn.linear_model import LinearRegression
```

```
In [29]: X = reg_data[["Rape"]]
y = reg_data[["UrbanPop"]]
```

```
In [30]: model = LinearRegression()
model.fit(X, y)
```

```

Out [30]: <style>.sk-global { /* Definition of color scheme common for light and dark mode */ --
sklearn-color-text: #000; --sklearn-color-text-muted: #666; --sklearn-color-line: gray; /*
Definition of color scheme for unfitted estimators */ --sklearn-color-unfitted-level-0:
#fff5e6; --sklearn-color-unfitted-level-1: #f6e4d2; --sklearn-color-unfitted-level-2:
#ffe0b3; --sklearn-color-unfitted-level-3: chocolate; /* Definition of color scheme for
fitted estimators */ --sklearn-color-fitted-level-0: #f0f8ff; --sklearn-color-fitted-level-1:
#d4ebff; --sklearn-color-fitted-level-2: #b3dbfd; --sklearn-color-fitted-level-3:
cornflowerblue; } .sk-global.light { /* Specific color for light theme */ --sklearn-color-text-
on-default-background: black; --sklearn-color-background: white; --sklearn-color-
border-box: black; --sklearn-color-icon: #696969; } .sk-global.dark { --sklearn-color-
text-on-default-background: white; --sklearn-color-background: #111; --sklearn-color-
border-box: white; --sklearn-color-icon: #878787; } .sk-global { color: var(--sklearn-
color-text); } .sk-global pre { padding: 0; } .sk-global input.sk-hidden--visually { border: 0;
clip-path: inset(100%); height: 1px; margin: -1px; overflow: hidden; padding: 0; position:
absolute; width: 1px; } .sk-global div.sk-dashed-wrapped { border: 1px dashed var(--
sklearn-color-line); margin: 0 0.4em 0.5em 0.4em; box-sizing: border-box; padding-
bottom: 0.4em; background-color: var(--sklearn-color-background); } .sk-global div.sk-
container { /* jupyter's `normalize.less` sets `[hidden] { display: none; }` but
bootstrap.min.css set `[hidden] { display: none !important; }` so we also need the
`!important` here to be able to override the default hidden behavior on the sphinx
rendered scikit-learn.org. See: https://github.com/scikit-learn/scikit-learn/issues/21755 */
display: inline-block !important; position: relative; } .sk-global div.sk-text-repr-fallback {
display: none; } div.sk-parallel-item, div.sk-serial, div.sk-item { /* draw centered vertical
line to link estimators */ background-image: linear-gradient(var(--sklearn-color-text-on-
default-background), var(--sklearn-color-text-on-default-background)); background-
size: 2px 100%; background-repeat: no-repeat; background-position: center center; } /*
Parallel-specific style estimator block */ .sk-global div.sk-parallel-item::after { content: "";
width: 100%; border-bottom: 2px solid var(--sklearn-color-text-on-default-background);
flex-grow: 1; } .sk-global div.sk-parallel { display: flex; align-items: stretch; justify-
content: center; background-color: var(--sklearn-color-background); position: relative; }
.sk-global div.sk-parallel-item { display: flex; flex-direction: column; } .sk-global div.sk-
parallel-item:first-child::after { align-self: flex-end; width: 50%; } .sk-global div.sk-
parallel-item:last-child::after { align-self: flex-start; width: 50%; } .sk-global div.sk-
parallel-item:only-child::after { width: 0; } /* Serial-specific style estimator block */ .sk-
global div.sk-serial { display: flex; flex-direction: column; align-items: center;
background-color: var(--sklearn-color-background); padding-right: 1em; padding-left:
1em; } /* Toggleable style: style used for estimator/Pipeline/ColumnTransformer box that
is clickable and can be expanded/collapsed. - Pipeline and ColumnTransformer use this
feature and define the default style - Estimators will overwrite some part of the style
using the `sk-estimator` class */ /* Pipeline and ColumnTransformer style (default) */ .sk-
global div.sk-toggleable { /* Default theme specific background. It is overwritten whether
we have a specific estimator or a Pipeline/ColumnTransformer */ background-color: var(--

```

```

sklearn-color-background); } /* Toggleable label */ .sk-global label.sk-toggleable__label {
cursor: pointer; display: flex; width: 100%; margin-bottom: 0; padding: 0.5em; box-sizing:
border-box; text-align: center; align-items: center; justify-content: center; gap: 0.5em; }
.sk-global label.sk-toggleable__label .caption { font-size: 0.6rem; font-weight: lighter;
color: var(--sklearn-color-text-muted); } .sk-global label.sk-toggleable__label-
arrow:before { /* Arrow on the left of the label */ content: "▸"; float: left; margin-right:
0.25em; color: var(--sklearn-color-icon); } .sk-global label.sk-toggleable__label-
arrow:hover:before { color: var(--sklearn-color-text); } /* Toggleable content - dropdown
*/ .sk-global div.sk-toggleable__content { display: none; text-align: left; /* unfitted */
background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-
toggleable__content.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-
level-0); } .sk-global div.sk-toggleable__content pre { margin: 0.2em; border-radius:
0.25em; color: var(--sklearn-color-text); /* unfitted */ background-color: var(--sklearn-
color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted pre { /* unfitted */
background-color: var(--sklearn-color-fitted-level-0); } .sk-global input.sk-
toggleable__control:checked~div.sk-toggleable__content { /* Expand drop-down */
display: block; width: 100%; overflow: visible; } .sk-global input.sk-
toggleable__control:checked~label.sk-toggleable__label-arrow:before { content: "▾"; } /*
Pipeline/ColumnTransformer-specific style */ .sk-global div.sk-label input.sk-
toggleable__control:checked~label.sk-toggleable__label { color: var(--sklearn-color-
text); background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-
label.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label {
background-color: var(--sklearn-color-fitted-level-2); } /* Estimator-specific style */ /*
Colorize estimator box */ .sk-global div.sk-estimator input.sk-
toggleable__control:checked~label.sk-toggleable__label { /* unfitted */ background-
color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted input.sk-
toggleable__control:checked~label.sk-toggleable__label { /* fitted */ background-color:
var(--sklearn-color-fitted-level-2); } .sk-global div.sk-label label.sk-toggleable__label,
.sk-global div.sk-label label { /* The background is the default theme color */ color: var(--
sklearn-color-text-on-default-background); } /* On hover, darken the color of the
background */ .sk-global div.sk-label:hover label.sk-toggleable__label { color: var(--
sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } /* Label
box, darken color on hover, fitted */ .sk-global div.sk-label.fitted:hover label.sk-
toggleable__label.fitted { color: var(--sklearn-color-text); background-color: var(--
sklearn-color-fitted-level-2); } /* Estimator label */ .sk-global div.sk-label label { font-
family: monospace; font-weight: bold; line-height: 1.2em; } .sk-global div.sk-label-
container { text-align: center; } /* Estimator-specific */ .sk-global div.sk-estimator { font-
family: monospace; border: 1px dotted var(--sklearn-color-border-box); border-radius:
0.25em; box-sizing: border-box; margin-bottom: 0.5em; /* unfitted */ background-color:
var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-estimator.fitted { /* fitted */
background-color: var(--sklearn-color-fitted-level-0); } /* on hover */ .sk-global div.sk-
estimator:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); }

```

```

.sk-global div.sk-estimator.fitted:hover { /* fitted */ background-color: var(--sklearn-
color-fitted-level-2); } /* Specification for estimator info (e.g. "i" and "?") */ /* Common
style for "i" and "?" */ .sk-estimator-doc-link, a:link.sk-estimator-doc-link, a:visited.sk-
estimator-doc-link { float: right; font-size: smaller; line-height: 1em; font-family:
monospace; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 1em;
height: 1em; width: 1em; text-decoration: none !important; margin-left: 0.5em; text-align:
center; /* unfitted */ border: var(--sklearn-color-unfitted-level-3) 1pt solid; color: var(--
sklearn-color-unfitted-level-3); } .sk-estimator-doc-link.fitted, a:link.sk-estimator-doc-
link.fitted, a:visited.sk-estimator-doc-link.fitted { /* fitted */ background-color: var(--
sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-3) 1pt solid; color:
var(--sklearn-color-fitted-level-3); } /* On hover */ div.sk-estimator:hover .sk-estimator-
doc-link:hover, .sk-estimator-doc-link:hover, div.sk-label-container:hover .sk-estimator-
doc-link:hover, .sk-estimator-doc-link:hover { /* unfitted */ background-color: var(--
sklearn-color-unfitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid;
color: var(--sklearn-color-unfitted-level-0); text-decoration: none; } div.sk-
estimator.fitted:hover .sk-estimator-doc-link.fitted:hover, .sk-estimator-doc-
link.fitted:hover, div.sk-label-container:hover .sk-estimator-doc-link.fitted:hover, .sk-
estimator-doc-link.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-
level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-color-
fitted-level-0); text-decoration: none; } /* Span, style for the box shown on hovering the
info icon */ .sk-estimator-doc-link span { display: none; z-index: 9999; position: relative;
font-weight: normal; right: .2ex; padding: .5ex; margin: .5ex; width: min-content; min-
width: 20ex; max-width: 50ex; color: var(--sklearn-color-text); box-shadow: 2pt 2pt 4pt
#999; /* unfitted */ background: var(--sklearn-color-unfitted-level-0); border: .5pt solid
var(--sklearn-color-unfitted-level-3); } .sk-estimator-doc-link.fitted span { /* fitted */
background: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-
3); } .sk-estimator-doc-link:hover span { display: block; } /* "?"-specific style due to the
`` HTML tag */ .sk-global a.estimator_doc_link { float: right; font-size: 1rem; line-height:
1em; font-family: monospace; background-color: var(--sklearn-color-unfitted-level-0);
border-radius: 1rem; height: 1rem; width: 1rem; text-decoration: none; /* unfitted */ color:
var(--sklearn-color-unfitted-level-1); border: var(--sklearn-color-unfitted-level-1) 1pt
solid; } .sk-global a.estimator_doc_link.fitted { /* fitted */ background-color: var(--
sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-1) 1pt solid; color:
var(--sklearn-color-fitted-level-1); } /* On hover */ .sk-global a.estimator_doc_link:hover {
/* unfitted */ background-color: var(--sklearn-color-unfitted-level-3); color: var(--
sklearn-color-background); text-decoration: none; } .sk-global
a.estimator_doc_link.fitted:hover { /* fitted */ background-color: var(--sklearn-color-
fitted-level-3); } .sk-top-container.sk-global { /* pydata-sphinx-theme hides overflow, so
scrolling is disabled. We need to set it to !important and add tabIndex="0" in the HTML to
allow keyboard-only users to navigate the display. */ overflow-x: scroll !important; max-
width: 100%; } .estimator-table { font-family: monospace; } .estimator-table summary {
padding: .5rem; cursor: pointer; } .estimator-table summary::marker { font-size: 0.7rem; }

```

```

.estimator-table details[open] { padding-left: 0.1rem; padding-right: 0.1rem; padding-
bottom: 0.3rem; } .estimator-table .parameters-table { margin-left: auto !important;
margin-right: auto !important; margin-top: 0; } .estimator-table .parameters-table tr:nth-
child(odd) { background-color: #fff; } .estimator-table .parameters-table tr:nth-
child(even) { background-color: #f6f6f6; } .estimator-table .parameters-table tr:hover td {
background-color: #e0e0e0; } .estimator-table table :is(td, th) { border: 1px solid
rgba(106, 105, 104, 0.232); } /* `table td` is set in notebook with right text-align. We need
to overwrite it. */ .estimator-table table td.param { text-align: left; position: relative;
padding: 0; } .user-set td { color:rgb(255, 94, 0); text-align: left !important; } .user-set
td.value { color:rgb(255, 94, 0); background-color: transparent; } .default td, .estimator-
table th { color: black; text-align: left !important; } .user-set td i, .default td i { color: black;
} td.fitted-att-type { white-space: preserve nowrap; } /* Styles for parameter
documentation links We need styling for visited so jupyter doesn't overwrite it */ a.param-
doc-link, a.param-doc-link:link, a.param-doc-link:visited { text-decoration: underline
dashed; text-underline-offset: .3em; color: inherit; display: block; padding: .5em; }
@supports(anchor-name: --doc-link) { a.param-doc-link, a.param-doc-link:link, a.param-
doc-link:visited { anchor-name: --doc-link; } } /* "hack" to make the entire area of the cell
containing the link clickable */ a.param-doc-link::before { position: absolute; content: "";
inset: 0; } .param-doc-description { display: none; position: absolute; z-index: 9999; left:
0; padding: .5ex; margin-left: 1.5em; color: var(--sklearn-color-text); box-shadow: .3em
.3em .4em #999; width: max-content; text-align: left; max-height: 10em; overflow-y:
auto; /* unfitted */ background: var(--sklearn-color-unfitted-level-0); border: thin solid
var(--sklearn-color-unfitted-level-3); } @supports(position-area: center right) { .param-
doc-description { position-area: center right; position: fixed; margin-left: 0; } } /* Fitted
state for parameter tooltips */ .fitted .param-doc-description { /* fitted */ background:
var(--sklearn-color-fitted-level-0); border: thin solid var(--sklearn-color-fitted-level-3); }
.param-doc-link:hover .param-doc-description { display: block; } .copy-paste-icon {
background-image:
url(data:image/svg+xml;base64,PHN2YyB4bWxucz0iaHR0cDovL3d3dy53My5vcmcvMjAwMk
background-repeat: no-repeat; background-size: 14px 14px; background-position: 0;
display: inline-block; width: 14px; height: 14px; cursor: pointer; } .features { font-family:
monospace; cursor: pointer; background-color: var(--sklearn-color-unfitted-level-0);
border: 1px dotted var(--sklearn-color-border-box); border-radius: .20em; margin-
bottom: 0.5em; font-size: inherit; /* Needed for jupyter */ } .features.fitted { background-
color: var(--sklearn-color-fitted-level-0); } .features.summary { cursor: pointer; display:
flex; margin-bottom: 0; text-align: center; align-items: center; justify-content: center;
gap: 0.5em; padding: .25em; } .features.details[open] > summary { color: var(--sklearn-
color-text); background-color: var(--sklearn-color-unfitted-level-2); border-radius:
.20em 0 0 0; } .features.fitted.details[open] > summary { background-color: var(--
sklearn-color-fitted-level-2); border-radius: .20em 0 0 0; } .features.details > summary
.arrow::before { content: "▶"; color: grey; } .features.details[open] > summary
.arrow::before { content: "▼"; } .features.details:hover > summary { margin: 0;

```

```
background-color: var(--sklearn-color-unfitted-level-2); } .features.fitted details:hover >
summary { margin: 0; background-color: var(--sklearn-color-fitted-level-2); } .features
.features-container { max-width: 15em; max-height: 10em; overflow: auto; scrollbar-
width: thin; padding: .25em 0.1rem; background-color: var(--sklearn-color-unfitted-level-
0); border-radius: 0 0 .5em .5em; } .features.fitted .features-container { background-
color: var(--sklearn-color-fitted-level-0); } .features .image-container { block-size: 1em;
inline-size: 1em; padding: 0; margin: 0%; display: flex; justify-content: center; align-
items: center; } .features .copy-paste-icon { background-size: 1em 1em; width: 1em;
height: 1em; filter: grayscale(100%) opacity(60%); } .features .features-container table {
width: 100%; margin: 0.01em; } .features .features-container table tr:nth-child(odd) {
background-color: #fff; } .features .features-container table tr:nth-child(even) {
background-color: #f6f6f6; } .features .features-container table tr:hover { background-
color: #e0e0e0; } .features .features-container table { table-layout: inherit; } .features
.features-container table td { text-align: left; padding: 0 0.5em; border: 1px solid
rgba(106, 105, 104, 0.232); white-space: nowrap; color: var(--sklearn-color-text); }
.total_features { display: flex; justify-content: center; margin-top: 0.5em; } </style>
<body>
```

LinearRegression()

**In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.**

```
<input class="sk-toggleable__control sk-hidden--visually sk-global" id="sk-estimator-id-
1" type="checkbox" checked><label for="sk-estimator-id-1" class="sk-toggleable__label
fitted sk-toggleable__label-arrow">
```

LinearRegression

[?Documentation for LinearRegression](#)

```
</label>
```

```
<details> <summary>Parameters</summary> <tbody> </tbody> <tbody></tbody>
```

```
fit_intercept fit_intercept: bool, default=True<br><br>Whether to calculate the
intercept for this model. If set<br>to False, no intercept will be used in
calculations<br>(i.e. data is expected to be centered). True
```

```
copy_X copy_X: bool, default=True<br><br>If True, X will be copied; else, it may be
overwritten. True
```

```
tol tol: float, default=1e-6<br><br>The precision of the solution ( `coef_` ) is
determined by `tol` which<br>specifies the convergence criterion of the underlying
solver. `tol` is<br>set as `atol` and `btol` of :func:`scipy.sparse.linalg.lsqr`
when<br>fitting on sparse training data. `tol` is set as `cond`
of<br>:func:`scipy.linalg.lstsq` when fitting on dense training data.<br><br>..
versionadded:: 1.7<br>.. versionchanged:: 1.9<br>Now supported on dense data,
interpreted as the `cond` parameter. 1e-06
```

```
n_jobs n_jobs: int, default=None<br><br>The number of jobs to use for the
computation. This will only provide<br>speedup in case of sufficiently large
problems, that is if firstly<br>`n_targets > 1` and secondly `X` is sparse or if
`positive` is set<br>to `True`. ``None`` means 1 unless in None
```

a
:obj: `joblib.parallel\_backend` context. ``-1`` means using all
processors.
See :term: `Glossary <n\_jobs>` for more details.

positive positive: bool, default=False

When set to ``True``, forces the coefficients to be positive. This
option is only supported for dense arrays.

For a comparison between a linear regression model with positive constraints
on the regression coefficients and a linear regression without such constraints,
see :ref: `sphx\_glr\_auto\_examples\_linear\_model\_plot\_nnls.py`.

.. versionadded:: 0.24

False

</details>

<details> <summary>Fitted attributes</summary> <tbody> <th>Name</th>

<th>Type</th> <th>Value</th> </tbody> <tbody></tbody>

coef_ coef_: array of shape (n_features,) or (n_targets, n_features)

Estimated coefficients for the linear regression problem.
If multiple targets are passed during the fit (y 2D), this
is a 2D array of shape (n_targets, n_features), while if only
one target is passed, this is a 1D array of length n_features.

ndarray[float64]
(1,)

[0.56]

feature_names_in_ feature_names_in_: ndarray of shape (n_features_in_,)

Names of features seen during :term: `fit`. Defined only when `X`
has feature names that are all strings.

.. versionadded:: 1.0

ndarray[object]
(1,)

['Rape']

intercept_ intercept_: float or array of shape (n_targets,)

Independent term in the linear model. Set to 0.0 if
`fit\_intercept = False`.

float64

54.11

n_features_in_ n_features_in_: int

Number of features seen during :term: `fit`.

.. versionadded:: 0.24

int

1

rank_ rank_: int

Rank of matrix `X`. Only available when `X` is dense.

int64

np.int64(1)

singular_ singular_: array of shape (min(X, y),)

Singular values of `X`. Only available when `X` is dense.

ndarray[float64]
(1,)

[57.87]

</details>

```
<script>/* Authors: The scikit-learn developers SPDX-License-Identifier: BSD-3-Clause */
function copyToClipboard(text, element) { // Get the parameter prefix from the closest
toggleable content const toggleableContent = element.closest('.sk-
toggleable__content'); const paramPrefix = toggleableContent ?
toggleableContent.dataset.paramPrefix : ''; const fullParamName = paramPrefix ? `
paramPrefix{text}` : text; const originalStyle = element.style; const computedStyle =
window.getComputedStyle(element); const originalWidth = computedStyle.width; const
originalHTML = element.innerHTML.replace('Copied!', '');
navigator.clipboard.writeText(fullParamName) .then(() => { element.style.width =
originalWidth; element.style.color = 'green'; element.innerHTML = "Copied!";
setTimeout(() => { element.innerHTML = originalHTML; element.style = originalStyle; },
2000); }) .catch(err => { console.error('Failed to copy:', err); element.style.color = 'red';
```

```

element.innerHTML = "Failed!"; setTimeout(() => { element.innerHTML = originalHTML;
element.style = originalStyle; }, 2000); }); return false; }
document.querySelectorAll('.copy-paste-icon').forEach(function(element) { const
toggleableContent = element.closest('.sk-toggleable__content'); const paramPrefix =
toggleableContent ? toggleableContent.dataset.paramPrefix : ''; const parent =
element.parentElement; if (!parent || !parent.nextElementSibling) {
console.warn('Expected copy-paste icon is missing from the DOM structure'); return; }
const paramName = element.parentElement.nextElementSibling.textContent.trim().split('
')[0]; const fullParamName = paramPrefix ? `paramPrefix{paramName}` :
paramName; element.setAttribute('title', fullParamName); }); /** * Copy the list of feature
names formatted as a Python list. * * @param {HTMLElement} element - The copy button
inside a `.features` block; its siblings * contain a `details` element and a table containing
feature named. * @returns {boolean} Always returns `false` so callers can prevent the
default click behavior. */ function copyFeatureNamesToClipboard(element) { var
detailsElem = element.closest('.features').querySelector('details'); var wasOpen =
detailsElem.open; detailsElem.open = true; var content =
element.closest('.features').querySelector('tbody').innerHTML.trim(); if (!wasOpen)
detailsElem.open = false; const rows = content.split('\n').map(row => `
row` `); const formattedText = `[${rows.join(',')}]\n`; const originalHTML =
element.innerHTML.replace('✓', ''); const originalStyle = element.style; const copyMark
= document.createElement('span'); copyMark.innerHTML = '✓'; copyMark.style.color =
'blue'; copyMark.style.fontSize = '1em'; navigator.clipboard.writeText(formattedText)
.then(() => { element.style.display = 'none';
element.parentElement.appendChild(copyMark); setTimeout(() => { copyMark.remove();
element.innerHTML = originalHTML; element.style = originalStyle; }, 1000); }).catch(err
=> { console.error('Failed to copy:', err); element.style.color = 'orange';
element.innerHTML = "Failed!"; setTimeout(() => { element.innerHTML = originalHTML;
element.style = originalStyle; }, 1000); }); return false; } /** * Adapted from Skrub *
https://github.com/skrub-
data/skrub/blob/403466d1d5d4dc76a7ef569b3f8228db59a31dc3/skrub/_reporting/_data/te
* @returns "light" or "dark" */ function detectTheme(element) { const body =
document.querySelector('body'); // Check VSCode theme const themeKindAttr =
body.getAttribute('data-vscode-theme-kind'); const themeNameAttr =
body.getAttribute('data-vscode-theme-name'); if (themeKindAttr && themeNameAttr) {
const themeKind = themeKindAttr.toLowerCase(); const themeName =
themeNameAttr.toLowerCase(); if (themeKind.includes("dark") ||
themeName.includes("dark")) { return "dark"; } if (themeKind.includes("light") ||
themeName.includes("light")) { return "light"; } } // Check Jupyter theme if
(body.getAttribute('data-jp-theme-light') === 'false') { return 'dark'; } else if
(body.getAttribute('data-jp-theme-light') === 'true') { return 'light'; } // Guess based on a
parent element's color const color = window.getComputedStyle(element.parentNode,
null).getPropertyValue('color'); const match = color.match(/^rgb\s*

```

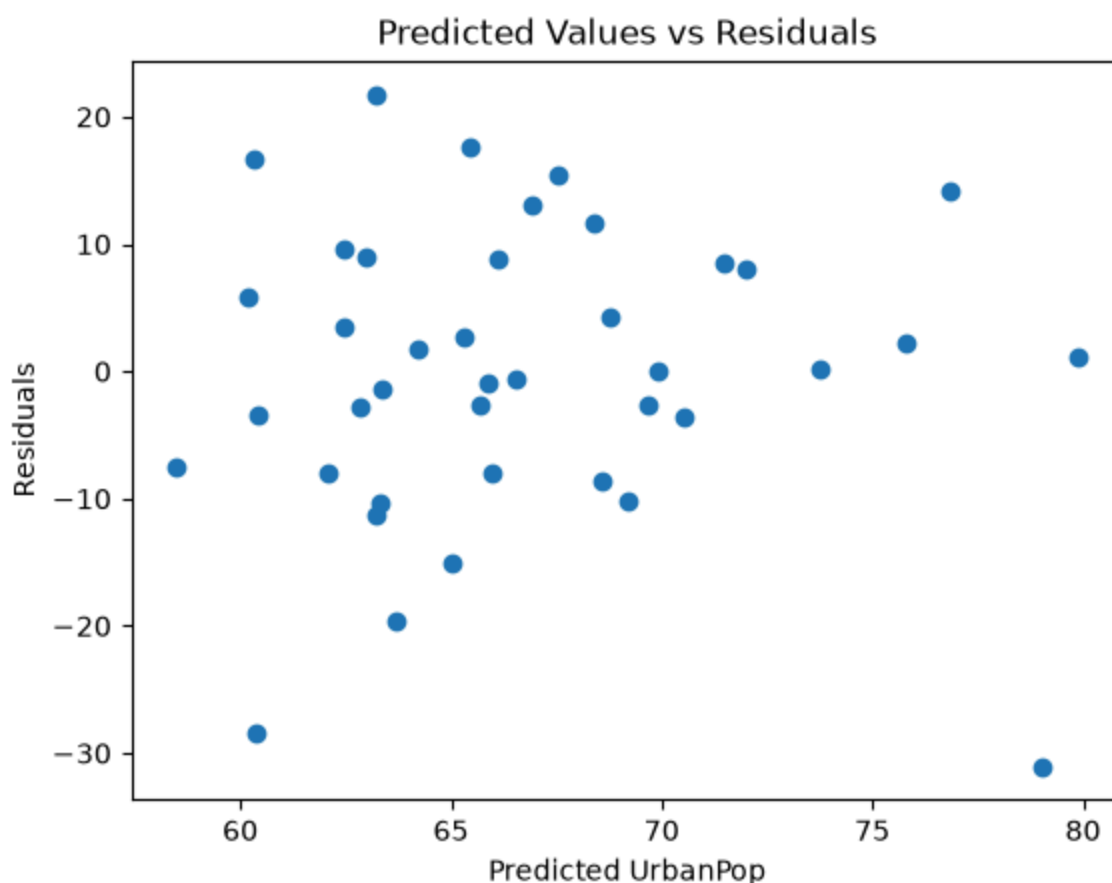
```

\s * (\d+)\s*, \s * (\d+)\s*, \s * (\d+)\s*\s*$ /i); if (match) { const [r, g, b] = [
parseFloat(match[1]), parseFloat(match[2]), parseFloat(match[3]) ]; //
https://en.wikipedia.org/wiki/HSL_and_HSV#Lightness const luma = 0.299 * r + 0.587 * g
+ 0.114 * b; if (luma > 180) { // If the text is very bright we have a dark theme return 'dark';
} if (luma < 75) { // If the text is very dark we have a light theme return 'light'; } //
Otherwise fall back to the next heuristic. } // Fallback to system preference return
window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light'; } function
forceTheme(elementId) { const estimatorElement =
document.querySelector(`#${elementId}`); if (estimatorElement === null) {
console.error(`Element with id ${elementId} not found.`); } else { const theme =
detectTheme(estimatorElement); estimatorElement.classList.add(theme); } }
forceTheme('sk-container-id-1');
```

```
In [31]: predicted = model.predict(X)
```

```
In [32]: residuals = y - predicted
```

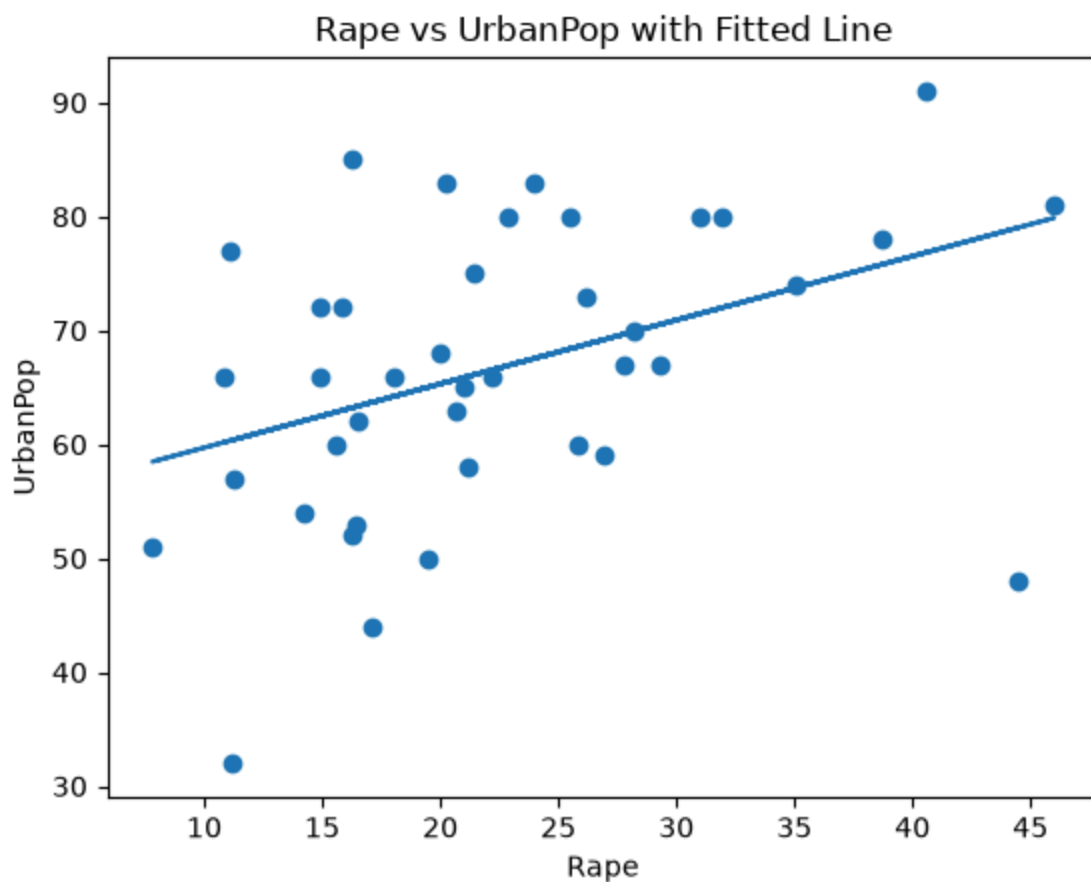
```
In [33]: plt.scatter(predicted, residuals)
plt.xlabel("Predicted UrbanPop")
plt.ylabel("Residuals")
plt.title("Predicted Values vs Residuals")
plt.show()
```



```
In [34]: plt.scatter(reg_data["Rape"], reg_data["UrbanPop"])

plt.plot(reg_data["Rape"], predicted)

plt.xlabel("Rape")
plt.ylabel("UrbanPop")
plt.title("Rape vs UrbanPop with Fitted Line")
plt.show()
```



```
In [35]: reg_no_outlier = reg_data[reg_data["State"] != "Alaska"]
X2 = reg_no_outlier[["Rape"]]
y2 = reg_no_outlier["UrbanPop"]
```

```
In [36]: model2 = LinearRegression()
model2.fit(X2, y2)
predicted2 = model2.predict(X2)
```

```
In [37]: plt.scatter(reg_data["Rape"], reg_data["UrbanPop"])

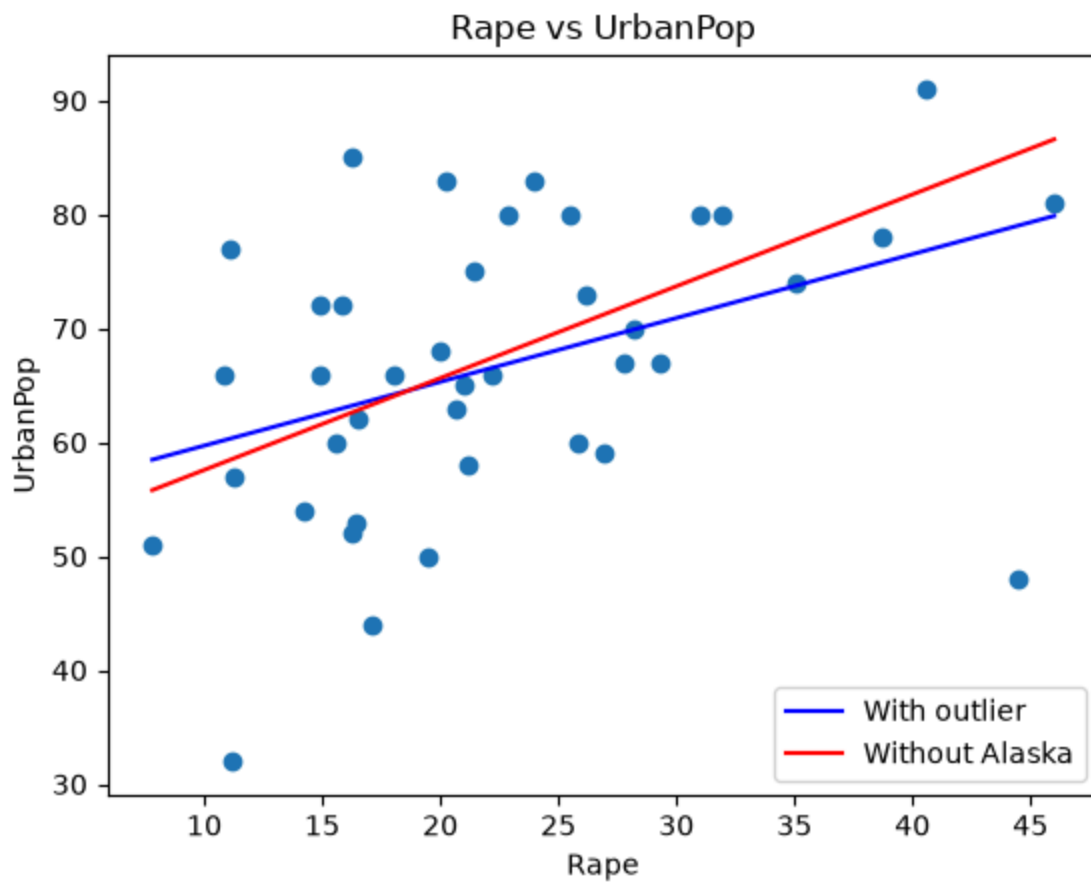
order1 = reg_data["Rape"].argsort()
plt.plot(
    reg_data["Rape"].iloc[order1],
    predicted[order1],
    color="blue",
    label="With outlier"
)
```

```

order2 = reg_no_outlier["Rape"].argsort()
plt.plot(
    reg_no_outlier["Rape"].iloc[order2],
    predicted2[order2],
    color="red",
    label="Without Alaska"
)

plt.xlabel("Rape")
plt.ylabel("UrbanPop")
plt.title("Rape vs UrbanPop")
plt.legend()
plt.show()

```



In []: